

```

import { describe, it, expect, vi, beforeEach } from "vitest";
import { render, screen, fireEvent } from "@testing-library/react";
import { Slider } from "../slider";

describe("Slider", () => {
  const defaultProps = {
    label: "Test Slider",
    min: 0,
    max: 100,
    step: 1,
    defaultValue: 50,
    onChange: vi.fn(),
  };

  beforeEach(() => {
    vi.clearAllMocks();
  });

  it("renders with label", () => {
    render(<Slider {...defaultProps} />);
    expect(screen.getByText("Test Slider")).toBeInTheDocument();
  });

  it("renders slider track and thumb", () => {
    render(<Slider {...defaultProps} />);
    const slider = screen.getByRole("slider");
    expect(slider).toBeInTheDocument();
    expect(slider).toHaveAttribute("aria-valuemin", "0");
    expect(slider).toHaveAttribute("aria-valuemax", "100");
    expect(slider).toHaveAttribute("aria-valuenow", "50");
  });

  it("calls onChange when value changes via keyboard", () => {
    render(<Slider {...defaultProps} />);
    const slider = screen.getByRole("slider");

    fireEvent.keyDown(slider, { key: "ArrowRight" });
    expect(defaultProps.onChange).toHaveBeenCalledWith(51);

    fireEvent.keyDown(slider, { key: "ArrowLeft" });
    expect(defaultProps.onChange).toHaveBeenCalledWith(50);
  });

  it("handles Home and End keys", () => {
    render(<Slider {...defaultProps} />);
    const slider = screen.getByRole("slider");

    fireEvent.keyDown(slider, { key: "Home" });
    expect(defaultProps.onChange).toHaveBeenCalledWith(0);
  });
});

```

```

    fireEvent.keyDown(slider, { key: "End" });
    expect(defaultProps.onChange).toHaveBeenCalledWith(100);
  });

  it("handles PageUp and PageDown keys", () => {
    render(<Slider {...defaultProps} />);
    const slider = screen.getByRole("slider");

    fireEvent.keyDown(slider, { key: "PageUp" });
    expect(defaultProps.onChange).toHaveBeenCalledWith(60);

    fireEvent.keyDown(slider, { key: "PageDown" });
    expect(defaultProps.onChange).toHaveBeenCalledWith(50);
  });

  it("displays value when showValue is true", () => {
    render(<Slider {...defaultProps} showValue={true} />);
    expect(screen.getByText("50")).toBeInTheDocument();
  });

  it("hides value when showValue is false", () => {
    render(<Slider {...defaultProps} showValue={false} />);
    expect(screen.queryByText("50")).not.toBeInTheDocument();
  });

  it("disables slider when disabled is true", () => {
    render(<Slider {...defaultProps} disabled={true} />);
    const slider = screen.getByRole("slider");
    expect(slider).toBeDisabled();
    expect(slider).toHaveAttribute("aria-disabled", "true");
  });

  it("shows error state when error prop is provided", () => {
    render(<Slider {...defaultProps} error="Invalid value" />);
    expect(screen.getByText("Invalid value")).toBeInTheDocument();
    const slider = screen.getByRole("slider");
    expect(slider).toHaveAttribute("aria-invalid", "true");
  });

  it("works with controlled value", () => {
    const { rerender } = render(<Slider {...defaultProps} value={75} />);
    const slider = screen.getByRole("slider");
    expect(slider).toHaveAttribute("aria-valuenow", "75");

    rerender(<Slider {...defaultProps} value={25} />);
    expect(screen.getByRole("slider")).toHaveAttribute("aria-valuenow", "25");
  });

  it("works with stepsArray (enum mode)", () => {
    const enumProps = {

```

```

    ...defaultProps,
    stepsArray: ["Low", "Medium", "High"],
    defaultValue: "Medium",
  });
  render(<Slider {...enumProps} />);
  const slider = screen.getByRole("slider");
  expect(slider).toHaveAttribute("aria-valuemin", "0");
  expect(slider).toHaveAttribute("aria-valuemax", "2");
  expect(slider).toHaveAttribute("aria-valuenow", "Medium");
  expect(screen.getByText("Medium")).toBeInTheDocument();
});

it("calls onChange with string value in enum mode", () => {
  const enumProps = {
    ...defaultProps,
    stepsArray: ["Low", "Medium", "High"],
    defaultValue: "Low",
  };
  render(<Slider {...enumProps} />);
  const slider = screen.getByRole("slider");

  fireEvent.keyDown(slider, { key: "ArrowRight" });
  expect(defaultProps.onChange).toHaveBeenCalledWith("Medium");
});

it("handles vertical orientation", () => {
  render(<Slider {...defaultProps} orientation="vertical" />);
  const slider = screen.getByRole("slider");
  expect(slider).toHaveAttribute("aria-orientation", "vertical");
});

it("clamps value to min/max bounds", () => {
  render(<Slider {...defaultProps} value={150} />);
  const slider = screen.getByRole("slider");
  expect(slider).toHaveAttribute("aria-valuenow", "100");

  render(<Slider {...defaultProps} value={-50} />);
  expect(screen.getByRole("slider")).toHaveAttribute("aria-valuenow", "0");
});

it("forwards ref to hidden input", () => {
  const ref = vi.fn();
  render(<Slider {...defaultProps} ref={ref} />);
  const hiddenInput = screen
    .getByRole("slider")
    .parentElement?.querySelector('input[type="hidden"]');
  expect(hiddenInput).toBeInTheDocument();
});

it("applies custom className", () => {

```

```

    render(<Slider {...defaultProps} className="custom-class" />);
    const track = screen.getByRole("slider").querySelector(".custom-class");
    expect(track).toBeInTheDocument();
  });

  it("handles step values correctly", () => {
    const stepProps = { ...defaultProps, step: 10, defaultValue: 0 };
    render(<Slider {...stepProps} />);
    const slider = screen.getByRole("slider");

    fireEvent.keyDown(slider, { key: "ArrowRight" });
    expect(stepProps.onChange).toHaveBeenCalledWith(10);

    fireEvent.keyDown(slider, { key: "ArrowRight" });
    expect(stepProps.onChange).toHaveBeenCalledWith(20);
  });

  it("renders progress bar with correct width", () => {
    render(<Slider {...defaultProps} defaultValue={50} />);
    const progressBar = screen
      .getByRole("slider")
      .querySelector('[style*="width: 50%"]');
    expect(progressBar).toBeInTheDocument();
  });
});

```